CSE 4057 Programming Assignment

Secure Zero-Trust File Drop System

Due: 24.05.2026, 23:59

Project goal: Design and implement a secure file drop system where users can upload encrypted files for other users through a server that is not trusted with plaintext file contents.

In this programming assignment, you will implement a simplified secure file drop system in which users can securely upload files for other users through a server. The system must be designed under a zero-trust storage assumption: the server may store and relay files, but it should not be trusted with plaintext file contents.

Your implementation must provide secure communication, user authentication, protected file upload/download, integrity verification, access control, expiration handling, and event logging. The goal is to help you gain practical experience in cryptography, network security, protocol design, and secure storage-oriented system design.

Prerequisite

It is assumed that you have basic knowledge on socket programming, symmetric and asymmetric cryptography, digital signatures, message authentication, and secure protocol design.

System Overview

- Clients: users who upload files for other users and retrieve files addressed to them.
- Server: stores encrypted files, manages upload/download requests, enforces access control, and maintains file status information.
- Certificate Authority (CA): a simple CA module that issues and signs certificates for clients and the server.

Zero-Trust Storage Assumption: The server may authenticate users, store encrypted files, maintain metadata, and enforce download rules, but it should not be able to read the actual plaintext contents of files.

Required Security Features

1) Public Key Generation and Certification

- Each client and the server must generate a public/private key pair.
- You must implement a simple Certificate Authority (CA) that verifies identities in a simplified way, issues certificates, and signs certificates with its own private key.
- Each client must obtain a certificate from the CA, and the server must also obtain a certificate from the CA.
- Certificates must be exchanged and validated during communication.
- You may use RSA or ECC for key generation and certificate signing.

2) Secure Client-Server Handshake and Session Key Establishment

- When a client connects to the server, they must perform a handshake.
- Each side must send a Hello message containing its certificate, a randomly generated nonce, and any additional parameters required by your protocol.
- After exchanging Hello messages, the client and server must establish a shared session secret using either RSA-encrypted random secret or ECDH (Elliptic Curve Diffie-Hellman).
- From the shared secret, derive the necessary keys using a proper key derivation mechanism such as HKDF.
- At minimum, derive one encryption key for client-to-server communication, one encryption key for server-to-client communication, and any additional values required by your design.

- You must clearly document your handshake and key derivation design in the README file.

3) Secure User Registration and Authentication

- The system must support multiple users.
- Each user and the server must own a CA-signed certificate.
- Before any secure upload, file listing, or retrieval operation, the client and server must authenticate each other using certificate-based authentication.
- As part of this process, each side must verify the other side's certificate, and each side must prove possession of the private key corresponding to that certificate.
- For this reason, sending only a certificate is not enough; your protocol must include an additional proof-of-possession step, such as signing a fresh nonce.
- The server must reject all sensitive requests from unauthenticated users, and the client must not continue if the server cannot be authenticated.

4) End-to-End Encrypted File Upload

- For each uploaded file, generate a fresh symmetric file encryption key.
- Encrypt the file using a secure symmetric encryption scheme such as AES-CBC or AES-CTR.
- Ensure that only the intended recipient can recover the file encryption key.
- A recommended design is to encrypt the file with a random symmetric key and then wrap that key using the recipient's public key.
- Alternatively, you may research and use an authenticated encryption mode such as AES-GCM.
- The file must not be uploaded or stored in plaintext.

5) Digital Signature for Origin Authentication

- For each uploaded file, the sender must generate a digital signature.
- The signature should cover appropriate data such as the sender identity, recipient identity, file identifier, file hash/checksum, timestamp, and expiration information if applicable.
- Upon upload, the server should verify the sender's signature and basic package consistency before accepting the file for storage.
- Upon retrieval, the recipient must verify the signature to confirm that the file was sent by the claimed sender and that it has not been modified.

6) Secure File Retrieval

- The recipient authenticates to the server.
- The server lists pending files for that recipient.
- The recipient downloads the encrypted file package.
- The recipient unwraps the file encryption key using their private key.
- The recipient decrypts the file and verifies the sender's signature.
- Unauthorized users must not be able to retrieve files intended for someone else.

7) Metadata and Access Control

- At minimum, maintain metadata such as file ID, sender ID, recipient ID, upload timestamp, status, and expiration time.
- Only the sender and authorized recipient should be able to query relevant file information.
- Only the intended recipient can retrieve the file.
- Unauthorized download attempts must be rejected and logged.

8) File Expiration

- Each file must have an expiration time.
- The server must check the expiration time before allowing retrieval.
- Expired files must not be retrievable.
- Expiration-related access attempts must be logged.
- Your README must explain how expiration is stored, checked, and enforced.

9) Replay Protection and Freshness

- Your protocol must include protections against replay attacks.
- At minimum, use one or more of the following: nonces, timestamps, sequence numbers, request IDs, or one-time tokens.
- You must explain where replay attacks may occur and how your system detects or prevents them.

10) Logging

- Your system must maintain log files.
- The logs should include connection establishment events, certificate verification results, file upload events, file retrieval events, failed authentication attempts, failed authorization attempts, signature verification failures, replay detection events, and expiration-related events.
- Logs must include timestamps and relevant entity information.
- Do not carelessly log sensitive plaintext secrets, private keys, or decrypted file contents.

Communication Protocol

You must design your own simple application-layer protocol for handshake, authentication, file upload, file listing, file retrieval, expiration handling, acknowledgements, and error reporting. The protocol should be clearly documented in the README file, including message types, field formats, step ordering, request/response matching, and replay protection logic.

Socket Programming

- You must use network sockets in your implementation. TCP is recommended.
- You may use multiple machines, or localhost on a single machine for testing.
- Your program should support at least one server, multiple clients, and multiple upload/download operations.
- A multi-threaded or asynchronous design is acceptable.

Language and Libraries

- You may use any high-level programming language such as Python, Java, or C++.
- You may use cryptographic libraries such as Python: cryptography, PyCryptodome; Java: javax.crypto and related standard libraries.
- You may not use ready-made secure channel protocols such as SSL/TLS libraries, OpenSSL-based secure socket wrappers, or similar high-level frameworks that directly solve the assignment for you.
- You must implement the core handshake, key establishment, encryption, signature verification, and protocol logic yourselves.

Group Work

- Groups of up to three students are allowed.
- In your README file, clearly declare how you communicated and coordinated as a team, the detailed division of labor, how you integrated and merged your code, and what each team member implemented.

Security Analysis

Even if your system includes several security mechanisms, security holes may still exist. You must include a short section in your README discussing possible vulnerabilities in your design, attack scenarios, and possible countermeasures. Example topics include man-in-the-middle attempts during handshake, replayed upload or download requests, unauthorized file access attempts, forged metadata, malicious server-side metadata modification, compromised client private keys, weak randomness, leakage through logs, and metadata leakage such as filenames, sizes, or timing information.

Minimum Functional Requirements Checklist

Feature	Status
CA creation and certificate issuance	Required
Secure client-server handshake and key derivation	Required
Mutual certificate-based authentication	Required
Encrypted file upload and encrypted storage	Required
Recipient-specific file-key protection	Required
Digital signature generation and verification	Required
Secure file listing and retrieval	Required
Access control enforcement	Required
File expiration enforcement	Required
Replay protection	Required
Event logging	Required
README documentation	Required

Bonus Features

Bonus points may be awarded for additional functionality such as the following. Bonus credit will be awarded based not only on whether a feature exists, but also on whether it is meaningfully implemented, enforced by the system rather than only described, and clearly documented in the README.

1) Revocation Before Download

- Allow the sender to revoke a file before it is successfully downloaded by the intended recipient.
- To receive bonus credit, revocation must be enforced by the server.
- A revoked file must not remain retrievable.
- Revocation must apply only before successful download.
- Revocation attempts and rejected retrieval attempts for revoked files must be logged.
- Your README must explain how revocation state is stored and checked.
- A simple user-interface level restriction is not sufficient; the server must actually enforce the revocation rule.

2) One-Time Download

- Allow a file to be downloaded only once by the intended recipient.
- To receive bonus credit, your design must define what counts as a successful retrieval.

- Only a successfully completed retrieval should consume the file.
- Interrupted or failed retrieval attempts should not automatically consume it.
- Any later retrieval attempt after a successful one-time download must be rejected.
- Repeated attempts must be logged.
- Your README must explain how interrupted, repeated, or concurrent retrieval attempts are handled.
- A simple boolean flag by itself is not sufficient for full bonus credit unless the surrounding retrieval logic is also meaningfully designed and documented.

3) Confidential Metadata

- Protect file metadata such as filename or description so that the server learns as little as possible.
- To receive bonus credit, at least some non-essential metadata must be protected from the server.
- Your design must clearly distinguish between metadata that must remain visible for routing/access control and metadata that can be hidden.
- Your README must explain exactly which metadata fields are protected and which remain visible.
- Simply omitting optional metadata is not sufficient; the bonus is for protecting metadata, not for avoiding it.

4) Large File Chunking

- Support chunk-based file transfer for large files while preserving integrity and correct ordering.
- To receive bonus credit, the file must be split into multiple chunks.
- The receiver must be able to reconstruct the original file correctly.
- Your design must include a way to detect missing, reordered, duplicated, or modified chunks.
- Your README must explain how chunk numbering, reassembly, and integrity checking are handled.
- Sending a file in multiple socket reads without an explicit chunking design is not sufficient.

5) Recipient Acknowledgement

- Allow the recipient to send a signed acknowledgement after successful download and verification.
- To receive bonus credit, the acknowledgement must be generated only after retrieval and verification.
- It must be cryptographically bound to the specific file (for example, file ID, timestamp, or status).
- It must be signed by the recipient.
- The sender or server must be able to verify it.
- A plain-text “received” message without signature protection is not sufficient.

6) End-to-End Encrypted Notes

- Allow the sender to include a short encrypted note together with the file.
- To receive bonus credit, the note must be protected so that unauthorized parties cannot read it.
- It must be bound to the same file package or transfer context.
- Your README must explain how the note is encrypted and how it is associated with the file.
- A plaintext comment field is not sufficient.

7) Containerized Deployment

- Package the system using Docker or a similar environment for easier testing.
- To receive bonus credit, the submission must include the necessary container configuration files.
- The system must be runnable with clearly documented steps.
- The containerized version must demonstrate the main system components in a usable way.
- Providing only a partial or non-working container setup is not sufficient.

Evaluation Rubric

Criterion	Points	What is evaluated
1) Public Key Infrastructure and Certificate Handling	15	Key generation, CA implementation, certificate issuance, exchange, and verification.
2) Secure Handshake and Session Key Establishment	15	Hello exchange, freshness, shared secret establishment, and key derivation.
3) Secure File Encryption and Upload	15	Correct file encryption, recipient-specific key protection, and secure upload workflow.
4) Digital Signature and Integrity Verification	15	Signature generation, server-side verification on upload, and recipient-side verification on retrieval.
5) Secure Retrieval and Access Control	10	Pending-file listing, recipient-only retrieval, and unauthorized access rejection.
6) File Expiration	10	Expiration enforcement, rejection of expired files, and proper logging/documentation.
7) Replay Protection and Freshness	5	Replay-defense mechanism and explanation in README.
8) Logging and Security-Aware Event Recording	5	Useful security logs without leaking sensitive data.
9) Protocol Design and Documentation	5	Clear protocol description, cryptographic choices, and processing steps.
10) Code Quality and Project Organization	3	Readable structure, comments, and ease of testing.
11) Security Analysis and Discussion of Limitations	2	Realistic limitations and proposed countermeasures.

Bonus points may be awarded in addition to the 100-point base score, at the instructor's discretion, for bonus features that are meaningfully implemented and documented.

Submission Details

- Submit a single .zip file containing the source code, README file, any configuration files, test instructions, sample test files, and sample screenshots or terminal outputs if helpful.
- The README must include the system design, protocol description, cryptographic choices, how certificates are issued and verified, handshake steps, how keys are derived, how files are encrypted and uploaded, how file

keys are protected for recipients, how signatures are created and verified, how access control and expiration are enforced, how replay protection is implemented, division of labor, identified security holes and proposed countermeasures, and how to run and test the system.

- The zip file name must include the full names of all group members.
- Submit your project via Google Classroom. Only one submission per group is allowed.

Demonstration / Testing Expectation

1. CA issues certificates.
2. Server starts and waits for connections.
3. Two or more clients connect and authenticate.
4. Client A uploads an encrypted file for Client B.
5. The server verifies the upload package and stores the encrypted package.
6. Client B lists available pending files.
7. Client B downloads the file.
8. Client B decrypts the file and verifies the sender's signature.
9. An unauthorized client attempts to access the file and is rejected.
10. An expired file is rejected if its expiration time has passed.
11. Logs show the relevant events.

Plagiarism Warning

- Your codes will be checked for plagiarism.
- Reusing code from previous years, online repositories, or other groups without proper authorization is prohibited.
- Any detected plagiarism will be severely penalized.
- You are expected to design and implement your own solution.